GitHub Actions — Category 3: Reusability & Modularity

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Category 2 (Actions — Building Blocks) assumed. Jenkins Shared Library experience assumed. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers (30s + Deep Dive) → Full YAML → Gotchas ⚠ → Connections

---

Table of Contents

- 3.1 Reusable Workflows — `workflow_call`, inputs, outputs, secrets
- 3.2 Calling Reusable Workflows from Other Repos
- 3.3 Composite Actions vs Reusable Workflows — When to Use Which ⚠
- 3.4 Workflow Templates — Org-Level `.github` Repo
- 3.5 Centralized Pipeline Management at Org Scale
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

3.1 Reusable Workflows — `workflow_call`, inputs, outputs, secrets

What It Is

A **reusable workflow** is a workflow file that exposes itself as callable by other workflows via the `workflow_call` event trigger. When called, it runs its own jobs on their own runners — completely independently from the caller's runner environment.

Think of it as a **subroutine at the pipeline level**: callers invoke it, pass typed inputs and secrets, receive outputs, and the reusable workflow handles everything internally.

Jenkins equivalent: A **Jenkins Shared Library pipeline template** — specifically a `vars/myPipeline.groovy` script that other Jenkinsfiles call with `myPipeline(config)`. The closest structural match is:

Jenkins Shared Library	GitHub Actions Reusable Workflow
`vars/deployPipeline.groovy`	`.github/workflows/deploy.yml` with `on: workflow_call:`
`config.environment` parameter	`inputs.environment`
`withCredentials([])` block	`secrets.my-secret` declaration

Jenkins Shared Library	GitHub Actions Reusable Workflow
Called via `deployPipeline(env: 'prod')`	Called via `uses: org/repo/.github/workflows/deploy.yml@main`
Runs on the same agent	Runs on its own declared runners

The key architectural difference from Jenkins: **the reusable workflow defines its own runners**. The caller has no control over where the jobs run — only what inputs to pass.

Why It Exists

Problem: You have 40 microservice repos. Each needs the same build → test → security scan → Docker push → deploy pipeline. Without reusable workflows:

- 40 copies of nearly identical workflow YAML
- A security hardening update requires 40 PRs
- Teams diverge in subtle ways — some pin action versions, some don't
- No way to enforce org-wide pipeline standards

Where Jenkins Shared Libraries fall short compared to reusable workflows:

- Jenkins Shared Libraries run in the calling Jenkinsfile's agent context — no runner isolation
- Shared Library updates can break all callers simultaneously if not versioned carefully
- No native typed input contract — usually a free-form `Map config` argument
- No built-in environment-level approval gates — must be implemented manually
- Visibility in the UI is merged into the calling pipeline, harder to trace

What reusable workflows add:

- Each invocation gets its own runner lifecycle — full isolation
- Typed inputs with descriptions, required flags, and defaults
- Declared secrets with explicit requirement statements
- Job-level outputs back to the caller
- The reusable workflow can declare its own `environment:` — triggering GitHub's native approval gates
- Full independent job entries in the caller's workflow run UI

How It Works Internally

```
Caller Workflow (app-repo)                Platform Workflows Repo
        |                                          |
        |  uses: org/platform/.github/             |
        |        workflows/deploy.yml@v3           |
        |  with:                                   |
        |    environment: production               |
        |    image-tag: abc123                     |
        |  secrets:                                |
```

```
        │    kubeconfig: ${{ secrets.KUBE }}       │
        │                                          │
        ▼                                          ▼
GitHub Orchestrator reads the reusable workflow's YAML
        │
        ▼
Resolves jobs declared in the reusable workflow
        │
        ├── Provisions runner(s) for each job (FROM reusable workflow's
runs-on)
        │
        ├── Injects inputs as   → inputs.<name> context
        ├── Injects secrets as  → secrets.<name> context
        │
        ├── Jobs run independently with their own runner lifecycle
        │
        └── Job outputs flow back → caller accesses via needs.
<job>.outputs.<key>
```

Execution model details:

- The reusable workflow's jobs appear **directly in the caller's workflow run UI** — they're not hidden behind a single step entry
- Inputs and secrets are resolved at the GitHub orchestrator level, before any runner is provisioned
- The reusable workflow cannot access the caller's full `secrets` context unless explicitly passed via `secrets: inherit` or individual secret declarations
- Caller and reusable workflow can be in **different repos** — GitHub handles cross-repo resolution at call time

---

Defining a Reusable Workflow — Complete Anatomy

```yaml
# .github/workflows/reusable-deploy.yml
# Stored in: my-org/platform-workflows repo

name: Reusable - Application Deploy

# ──────────────────────────────────────────────────────────
# THE TRIGGER THAT MAKES THIS REUSABLE
# ──────────────────────────────────────────────────────────
on:
  workflow_call:

    # ── INPUTS ──────────────────────────────────────
    inputs:
      environment:
        description: 'Target environment (staging | production)'
        required: true
        type: string            # string | boolean | number
```

```yaml
      image-tag:
        description: 'Docker image tag to deploy'
        required: true
        type: string

      helm-release-name:
        description: 'Helm release name override'
        required: false
        type: string
        default: 'app'

      dry-run:
        description: 'Run without making changes'
        required: false
        type: boolean
        default: false

      replicas:
        description: 'Number of pod replicas'
        required: false
        type: number
        default: 2

    # ── SECRETS ──────────────────────────────────────────
    secrets:
      kubeconfig-base64:
        description: 'Base64-encoded kubeconfig for the target cluster'
        required: true
      slack-webhook:
        description: 'Slack webhook URL for notifications'
        required: false   # Optional secret — must guard access with if:

    # ── OUTPUTS ──────────────────────────────────────────
    outputs:
      deployed-image:
        description: 'Full image:tag string that was deployed'
        value: ${{ jobs.deploy.outputs.deployed-image }}
      helm-revision:
        description: 'Helm release revision number after deploy'
        value: ${{ jobs.deploy.outputs.helm-revision }}
      deploy-status:
        description: 'success | failure'
        value: ${{ jobs.deploy.outputs.status }}

# ──────────────────────────────────────────────────────────
# JOBS — these run on their OWN runners
# ──────────────────────────────────────────────────────────
jobs:

  # ── JOB 1: Pre-deploy validation ────────────────────
  validate:
    name: Validate deploy parameters
    runs-on: ubuntu-latest
    steps:
```

```yaml
      - name: Validate inputs
        run: |
          VALID_ENVS=("staging" "production" "dev")
          ENV="${{ inputs.environment }}"
          if [[ ! " ${VALID_ENVS[@]} " =~ " ${ENV} " ]]; then
            echo "::error::Invalid environment '${ENV}'. Valid:
${VALID_ENVS[*]}"
            exit 1
          fi
          echo "✅ Environment '${ENV}' is valid"

  # ── JOB 2: The actual deploy ──────────────────────────
  deploy:
    name: Deploy to ${{ inputs.environment }}
    runs-on: ubuntu-latest
    needs: validate

    # ⚠ The reusable workflow ITSELF declares the environment
    # This triggers GitHub's native approval gates
    environment: ${{ inputs.environment }}

    # Job-level outputs — these flow back to the caller
    outputs:
      deployed-image: ${{ steps.deploy.outputs.image }}
      helm-revision:  ${{ steps.deploy.outputs.revision }}
      status:         ${{ steps.set-status.outputs.status }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2

      # Access the secret passed by the caller
      - name: Configure kubeconfig
        run: |
          mkdir -p ~/.kube
          echo "${{ secrets.kubeconfig-base64 }}" | base64 -d >
~/.kube/config
          chmod 600 ~/.kube/config

      - name: Setup Helm
        uses: azure/setup-helm@b9e51907a09c216f16ebe8536097933489208112  #
v4
        with:
          version: '3.14.0'

      - name: Deploy via Helm
        id: deploy
        run: |
          DRY_RUN_FLAG=""
          if [ "${{ inputs.dry-run }}" = "true" ]; then
            DRY_RUN_FLAG="--dry-run"
            echo "::notice::DRY RUN — no changes will be applied"
          fi
```

```
          helm upgrade --install \
            ${{ inputs.helm-release-name }} \
            ./helm/${{ inputs.helm-release-name }} \
            --namespace ${{ inputs.environment }} \
            --create-namespace \
            --set image.tag=${{ inputs.image-tag }} \
            --set replicaCount=${{ inputs.replicas }} \
            --wait \
            --timeout 5m \
            --atomic \
            ${DRY_RUN_FLAG}

          REVISION=$(helm history ${{ inputs.helm-release-name }} \
            --namespace ${{ inputs.environment }} \
            --max 1 \
            --output json | jq -r '.[0].revision')

          echo "image=${{ inputs.image-tag }}" >> $GITHUB_OUTPUT
          echo "revision=${REVISION}" >> $GITHUB_OUTPUT

      - name: Verify rollout
        if: inputs.dry-run == false
        run: |
          kubectl rollout status \
            deployment/${{ inputs.helm-release-name }} \
            --namespace ${{ inputs.environment }} \
            --timeout=5m

      - name: Set status output
        id: set-status
        if: always()
        run: |
          if [ "${{ job.status }}" = "success" ]; then
            echo "status=success" >> $GITHUB_OUTPUT
          else
            echo "status=failure" >> $GITHUB_OUTPUT
          fi

  # ── JOB 3: Post-deploy notification ──────────────────
  notify:
    name: Notify deployment status
    runs-on: ubuntu-latest
    needs: deploy
    if: always() && secrets.slack-webhook != ''

    steps:
      - name: Send Slack notification
        run: |
          STATUS="${{ needs.deploy.result }}"
          EMOJI=$([[ "$STATUS" == "success" ]] && echo "✅" || echo "❌")

          curl -s -X POST "${{ secrets.slack-webhook }}" \
            -H 'Content-Type: application/json' \
            -d "{
```

```
                \"text\": \"${EMOJI} Deploy to *${{ inputs.environment }}* -
${STATUS}\",
                \"attachments\": [{
                  \"color\": \"$([[ \"$STATUS\" == \"success\" ]] && echo
\"good\" || echo \"danger\")\",
                  \"fields\": [
                    {\"title\": \"Image Tag\", \"value\": \"\`${{
inputs.image-tag }}\`\", \"short\": true},
                    {\"title\": \"Environment\", \"value\": \"${{
inputs.environment }}\", \"short\": true}
                  ]
                }]
              }"
```

Input Types — Complete Reference

```yaml
on:
  workflow_call:
    inputs:
      # STRING — most common
      environment:
        type: string
        required: true
        description: 'Target environment name'

      # BOOLEAN — only true/false
      dry-run:
        type: boolean
        required: false
        default: false
        description: 'Simulate without making changes'

      # NUMBER — integer or float
      replicas:
        type: number
        required: false
        default: 2
        description: 'Pod replica count'

    secrets:
      # Secrets have no type — always treated as opaque strings
      api-key:
        required: true
        description: 'API key for deployment system'
      optional-token:
        required: false    # Optional secret — must guard usage with if:
```

Accessing Inputs and Secrets in the Reusable Workflow

```yaml
jobs:
  my-job:
    runs-on: ubuntu-latest
    steps:
      # Inputs — accessed via inputs context
      - run: |
          echo "Environment: ${{ inputs.environment }}"
          echo "Dry run: ${{ inputs.dry-run }}"
          echo "Replicas: ${{ inputs.replicas }}"

      # Boolean input in shell condition
      - if: inputs.dry-run == true
        run: echo "This is a dry run"

      # Secrets — accessed via secrets context
      - run: ./deploy.sh
        env:
          API_KEY: ${{ secrets.api-key }}

      # Checking if optional secret was provided
      - if: secrets.optional-token != ''
        run: echo "Token was provided"
```

Outputs from Reusable Workflows — The Full Chain

```yaml
# Reusable workflow — how outputs flow UP from step → job → workflow_call
outputs
on:
  workflow_call:
    outputs:
      # workflow_call output references a JOB output
      build-id:
        value: ${{ jobs.build.outputs.id }}    # Must be jobs.
<id>.outputs.<key>
        description: 'Unique build identifier'

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      # Job output references a STEP output
      id: ${{ steps.generate.outputs.build-id }}

    steps:
      - name: Generate ID
        id: generate
        run: echo "build-id=$(uuidgen)" >> $GITHUB_OUTPUT
```

```yaml
# Caller workflow — accessing the reusable workflow's outputs
jobs:
  call-build:
    uses: my-org/platform/.github/workflows/build.yml@main
    with:
      image-tag: ${{ github.sha }}

  use-outputs:
    needs: call-build
    runs-on: ubuntu-latest
    steps:
      # Access via needs.<calling-job-id>.outputs.<workflow-output-name>
      - run: echo "Build ID: ${{ needs.call-build.outputs.build-id }}"
```

Real-World Production Example: Full CI/CD Using Reusable Workflows

```yaml
# app-repo/.github/workflows/main-pipeline.yml
name: Application CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:

  # ── CI Phase ──────────────────────────────────────────
  ci:
    uses: my-org/platform-workflows/.github/workflows/reusable-ci.yml@v3
    with:
      node-version: '20'
      run-e2e-tests: ${{ github.ref == 'refs/heads/main' }}
    secrets:
      sonar-token: ${{ secrets.SONAR_TOKEN }}
      npm-token: ${{ secrets.NPM_PUBLISH_TOKEN }}

  # ── Build Docker Image ────────────────────────────────
  build-image:
    needs: ci
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    uses: my-org/platform-workflows/.github/workflows/reusable-docker-
build.yml@v3
    with:
      image-name: payment-api
      registry: ghcr.io/my-org
      tag: ${{ github.sha }}
      platforms: linux/amd64,linux/arm64
    secrets:
```

```
      registry-token: ${{ secrets.GITHUB_TOKEN }}

  # ── Deploy to Staging ──────────────────────────────
  deploy-staging:
    needs: build-image
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@v3
    with:
      environment: staging
      image-tag: ${{ github.sha }}
      helm-release-name: payment-api
      dry-run: false
      replicas: 2
    secrets:
      kubeconfig-base64: ${{ secrets.STAGING_KUBECONFIG_B64 }}
      slack-webhook: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}

  # ── Integration Tests Against Staging ──────────────
  integration-tests:
    needs: deploy-staging
    uses: my-org/platform-workflows/.github/workflows/reusable-
integration-tests.yml@v3
    with:
      target-url: https://payment-api.staging.my-org.com
      test-suite: smoke
    secrets:
      test-api-key: ${{ secrets.STAGING_TEST_API_KEY }}

  # ── Deploy to Production (requires manual approval) ──
  deploy-production:
    needs: [build-image, integration-tests]
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@v3
    with:
      environment: production          # Production env has required
reviewers
      image-tag: ${{ github.sha }}
      helm-release-name: payment-api
      dry-run: false
      replicas: 5
    secrets:
      kubeconfig-base64: ${{ secrets.PROD_KUBECONFIG_B64 }}
      slack-webhook: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}

  # ── Report final status ────────────────────────────
  pipeline-summary:
    needs: [deploy-staging, integration-tests, deploy-production]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - name: Write pipeline summary
        run: |
          cat >> $GITHUB_STEP_SUMMARY << EOF
          ## 🚀 Pipeline Summary
```

```
          | Stage | Status |
          |-------|--------|
          | CI | ${{ needs.ci.result }} |
          | Deploy Staging | ${{ needs.deploy-staging.result }} |
          | Integration Tests | ${{ needs.integration-tests.result }} |
          | Deploy Production | ${{ needs.deploy-production.result }} |

          **Image:** \`${{ github.sha }}\`
          **Deployed image:** \`${{ needs.deploy-
production.outputs.deployed-image }}\`
          EOF
```

Interview Answer (30–45 seconds)

> "A reusable workflow exposes itself via the `workflow_call` trigger. Other workflows call it at the job level using `uses:`, passing typed inputs via `with:` and secrets via `secrets:`. The reusable workflow runs its own jobs on its own runners — it's not borrowing the caller's execution context. It can declare its own `environment:` on jobs, which triggers GitHub's approval gates. Outputs flow back through a three-level chain: step output → job output → workflow_call output → caller reads via `needs.<job>.outputs.<key>`. This is the mechanism for standardizing pipelines across 40+ repos — one reusable deploy workflow, 40 callers, one place to update."

Deep Dive: Secrets Inheritance

```
# Option 1: Pass secrets individually (explicit, auditable)
jobs:
  call:
    uses: org/repo/.github/workflows/deploy.yml@main
    secrets:
      kubeconfig: ${{ secrets.PROD_KUBECONFIG }}
      slack-token: ${{ secrets.SLACK_TOKEN }}

# Option 2: Inherit ALL caller secrets (convenient, less secure)
jobs:
  call:
    uses: org/repo/.github/workflows/deploy.yml@main
    secrets: inherit
    # The reusable workflow gets access to ALL secrets the caller has
access to
    # Useful but reduces auditability — harder to know which secrets are
used
```

`secrets: inherit` **gotcha:** The reusable workflow can only access secrets that its own `workflow_call.secrets` block declares **when using individual secret passing**. With `inherit`, it gets everything — but the reusable workflow still needs to declare secrets in its `workflow_call.secrets`

section for them to be accessible via `${{ secrets.name }}`. Without the declaration, `inherit` silently passes them but the reusable workflow can't reference them by name.

Common Interview Questions

Q: Can a reusable workflow call another reusable workflow?

> "Yes — reusable workflows can be nested, but only up to **4 levels deep**. Each level counts as one. Caller → Reusable A → Reusable B → Reusable C is the maximum before GitHub blocks it. In practice, deeper than 2 levels becomes hard to debug. The nesting limit exists to prevent infinite loops and runaway job spawning."

Q: Can a reusable workflow use `strategy.matrix`?

> "Yes — reusable workflows can define their own matrix strategy internally. The caller has no visibility into or control over that matrix. If you need to run a matrix that varies based on caller inputs, the reusable workflow reads the input and uses it in the matrix definition using `fromJSON()`. For example, the caller passes `node-versions: '[\"18\",\"20\",\"22\"]'` as a string input, and the reusable workflow does `matrix: node: ${{ fromJSON(inputs.node-versions) }}`."

Gotchas ⚠

- **`jobs:` is the minimum required content in a reusable workflow.** A `workflow_call` workflow with no jobs is invalid and will error.
- **Reusable workflows can only reference secrets via `${{ secrets.<name> }}`** — not via `env:` at the workflow level. Secrets must be passed into steps via `env:` at the step level.
- **Inputs in reusable workflows are immutable** — a job in the reusable workflow cannot modify an input value. If you need to transform an input, do it in a step and capture the result as a step output.
- **`workflow_call` cannot be combined with other triggers in a way that's intuitive.** A workflow with both `push:` and `workflow_call:` will run under the push trigger when a push happens, and under workflow_call when called. Be careful with secrets access — the push trigger may not have the same secrets as the caller.
- **The caller's `permissions:` do NOT pass to the reusable workflow.** The reusable workflow must declare its own permissions. If the reusable workflow needs `id-token: write` for OIDC, it must declare that itself.
- **Reusable workflow jobs show up in the caller's run UI directly** — not nested under a single collapsible entry. This is actually a feature (transparency) but can make complex callers with many reusable calls visually noisy.
- **Optional secrets cannot be directly checked with `if: secrets.foo != ''` at the job level** in older GitHub Actions behavior. Use a step-level `if:` or handle the empty string in your script.

3.2 Calling Reusable Workflows from Other Repos

What It Is

Reusable workflows can live in a completely separate repository from the callers — enabling a **centralized platform team** to own and maintain shared pipelines while product teams consume them.

Jenkins equivalent: Keeping your Shared Libraries in a dedicated repo (e.g., `my-org/jenkins-shared-libraries`) and configuring Jenkins to load them globally. The difference: with GitHub Actions, there's no central server configuration needed — any workflow can reference any public (or accessible private) workflow file by its repo path.

Why Cross-Repo Reusable Workflows?

The organizational model it enables:

```
my-org/
  platform-workflows/          ← Platform team owns this
    .github/
      workflows/
        reusable-ci.yml
        reusable-docker-build.yml
        reusable-deploy.yml
        reusable-security-scan.yml
        reusable-terraform.yml

  payment-api/                 ← Product team owns this
    .github/workflows/
      main.yml                 ← Calls platform workflows

  user-service/                ← Another product team
    .github/workflows/
      main.yml                 ← Calls same platform workflows

  inventory-service/
    .github/workflows/
      main.yml
```

This gives the platform team:

- Single source of truth for all pipeline logic
- Ability to push security and compliance fixes to all repos at once
- Ability to version the platform workflows independently of product repos
- Full audit trail of which version each repo is running

Access Rules for Cross-Repo Reusable Workflows

Caller repo type	Reusable workflow repo type	Works?	Notes
Public	Public	✅	Always works
Private	Same org, Private	✅	Default behavior

Caller repo type	Reusable workflow repo type	Works?	Notes
Private	Different org, Private	❌	Cannot access
Private	Different org, Public	✅	Public repos accessible
Internal (GHES)	Same enterprise	✅	With policy enabled
Any	Same repo	✅	Always works

Restricting who can call your reusable workflow:

GitHub does not natively provide caller-level restrictions on reusable workflows. However you can:

1. Keep the reusable workflow repo private (only same-org repos can call it)
2. Use `CODEOWNERS` + branch protection on the platform repo to control who changes the workflows
3. Use `required_workflows` (GitHub Enterprise) to enforce specific workflow usage

---

Syntax for Cross-Repo Calls

```
jobs:
  # ── Same repo ──────────────────────────────────────────────
  local-call:
    uses: ./.github/workflows/reusable.yml
    # No @ref — uses current branch's version of the workflow

  # ── Different repo, same org — with version pinning ──────────
  deploy:
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@v3
    #     ^org   ^repo              ^path to workflow file
^ref

  # ── Pinned to specific SHA (recommended for production) ──────
  deploy-secure:
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@a3f8c2d1e4b5
    #                                                          ↑
commit SHA

  # ── Pinned to branch (floating — useful for dev/test only) ───
  deploy-dev:
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@main

  # ── Passing inputs and secrets ───────────────────────────────
  deploy-with-data:
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@v3
    with:
      environment: production
      image-tag: ${{ github.sha }}
```

```
    secrets:
      kubeconfig-base64: ${{ secrets.PROD_KUBECONFIG_B64 }}

  # ── secrets: inherit — pass ALL caller secrets ────────────────
  deploy-inherit:
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@v3
    with:
      environment: staging
    secrets: inherit
```

Versioning Strategy for Platform Workflows

The same semantic versioning convention as actions applies:

```
# In the platform-workflows repo — release process
git tag -fa v3 -m "Update v3 to include OIDC support"
git tag -a v3.1.0 -m "Release v3.1.0"
git push origin v3 --force
git push origin v3.1.0

# Callers:
# Stability-first:
uses: my-org/platform-workflows/.github/workflows/deploy.yml@v3.1.0

# Auto-update within major version:
uses: my-org/platform-workflows/.github/workflows/deploy.yml@v3

# Maximum security (no silent updates):
uses: my-org/platform-workflows/.github/workflows/deploy.yml@a3f8c2d1
```

Dependabot for cross-repo reusable workflow versions:

```
# caller-repo/.github/dependabot.yml
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
```

Dependabot detects `uses:` references to external reusable workflows and opens PRs to update them — same as it does for action pinning.

Real-World: Platform Workflow with Caller Metadata

```yaml
# platform-workflows/.github/workflows/reusable-ci.yml
name: Reusable — CI Pipeline

on:
  workflow_call:
    inputs:
      service-name:
        description: 'Name of the calling service — used for reporting'
        required: true
        type: string
      node-version:
        description: 'Node.js version'
        required: false
        type: string
        default: '20'
      run-e2e:
        description: 'Whether to run E2E tests'
        required: false
        type: boolean
        default: false
      coverage-threshold:
        description: 'Minimum test coverage percentage'
        required: false
        type: number
        default: 80
    secrets:
      sonar-token:
        required: true
      npm-read-token:
        required: false
    outputs:
      coverage-pct:
        description: 'Actual test coverage percentage achieved'
        value: ${{ jobs.test.outputs.coverage }}
      sonar-status:
        description: 'SonarCloud quality gate status'
        value: ${{ jobs.sonar.outputs.gate-status }}

jobs:
  lint:
    name: Lint (${{ inputs.service-name }})
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4
      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
# v4
        with:
          node-version: ${{ inputs.node-version }}
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
```

```
  test:
    name: Unit Tests (${{ inputs.service-name }})
    runs-on: ubuntu-latest
    needs: lint
    outputs:
      coverage: ${{ steps.coverage.outputs.pct }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
        with:
          node-version: ${{ inputs.node-version }}
          cache: npm
      - run: npm ci
      - name: Run tests with coverage
        run: npm test -- --coverage --coverageReporters=json-summary
      - name: Extract coverage percentage
        id: coverage
        run: |
          PCT=$(jq '.total.lines.pct' coverage/coverage-summary.json)
          echo "pct=${PCT}" >> $GITHUB_OUTPUT
          echo "Coverage: ${PCT}%"
          # Enforce threshold
          THRESHOLD=${{ inputs.coverage-threshold }}
          if (( $(echo "$PCT < $THRESHOLD" | bc -l) )); then
            echo "::error::Coverage ${PCT}% is below threshold
${THRESHOLD}%"
            exit 1
          fi

  sonar:
    name: SonarCloud Analysis (${{ inputs.service-name }})
    runs-on: ubuntu-latest
    needs: test
    outputs:
      gate-status: ${{ steps.sonar.outputs.status }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        with:
          fetch-depth: 0
      - name: SonarCloud Scan
        id: sonar
        uses: SonarSource/sonarcloud-github-
action@49e6cd3b187936a73b8280d59be7e75d3f8cc39  # v2
        env:
          GITHUB_TOKEN: ${{ github.token }}
          SONAR_TOKEN: ${{ secrets.sonar-token }}
        with:
          args: >
            -Dsonar.projectKey=my-org_${{ inputs.service-name }}
            -Dsonar.organization=my-org
      - run: |
          STATUS=$(cat .scannerwork/report-task.txt | grep ceTaskUrl | cut
-d= -f2-)
```

```
            echo "status=passed" >> $GITHUB_OUTPUT

    e2e:
      name: E2E Tests (${{ inputs.service-name }})
      runs-on: ubuntu-latest
      needs: test
      if: inputs.run-e2e == true
      steps:
        - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        - run: npm run test:e2e
```

Interview Answer (30–45 seconds)

> "Cross-repo reusable workflows are how platform teams centralize pipeline logic. The reusable workflow lives in a dedicated platform repo — say `my-org/platform-workflows` — and product repos call it with `uses: my-org/platform-workflows/.github/workflows/deploy.yml@v3`. Access is automatic for repos in the same GitHub org for private repos. For production callers, you pin to a specific commit SHA or immutable version tag. Dependabot can manage version bumps automatically. The platform team owns one repo, pushes fixes once, and every calling repo benefits. The key organizational benefit is that compliance, security scanning, and deployment standards live in one place — not duplicated across every service repo."

Gotchas ⚠

- **The `@ref` in a cross-repo `uses:` is the ref of the reusable workflow's repo**, not the caller's repo. If you write `uses: org/platform/.github/workflows/deploy.yml@main`, it resolves `main` in the `platform` repo.
- **`uses:` and `runs-on:` cannot coexist at the job level.** A job either calls a reusable workflow (`uses:`) or declares steps with `runs-on:`. Not both.
- **A reusable workflow job cannot have `steps:`.** The entire job body is replaced by the `uses:` call. You can have a separate downstream job with steps that reads the outputs.
- **Cross-repo reusable workflows from public repos are available to everyone.** If you put sensitive logic in a public reusable workflow (even without secrets), anyone can read and potentially exploit the workflow logic.
- **GitHub Enterprise Server (GHES) requires explicit org-level policy** to allow cross-org or cross-enterprise reusable workflow calls. It's not enabled by default.

3.3 Composite Actions vs Reusable Workflows — When to Use Which ⚠

What It Is

This is the most commonly confused distinction in GitHub Actions and is tested in nearly every senior-level interview. Both enable reuse, but at fundamentally different levels of the execution model.

The Core Distinction — One Sentence Each

- **Composite Action:** Reuses a *sequence of steps*, running inline on the **caller's job's runner**. Called at the **step** level.
- **Reusable Workflow:** Reuses an *entire job or multi-job pipeline*, each job running on **its own runner**. Called at the **job** level.

Side-by-Side Comparison — Full Detail

Dimension	Composite Action	Reusable Workflow
Called at	Step level (`uses:` inside `steps:`)	Job level (`uses:` at job level, no `steps:`)
Runs on	Caller's existing runner	Its own runner(s) — defined in the reusable workflow
Runner lifecycle	No new runner created	New runner provisioned per job
Isolation	Shares caller's env, workspace, filesystem	Fully isolated runners
Can define `services:`	❌ No	✅ Yes
Can define `matrix:`	❌ No	✅ Yes
Can target `environment:`	❌ No	✅ Yes (with approval gates)
Can set `permissions:`	❌ No	✅ Yes, per-job
Can set `concurrency:`	❌ No	✅ Yes, per-job
Secret access	Via `inputs:` only	Via `secrets:` declaration or `inherit`
Output declaration	`value: ${{ steps.x.outputs.y }}`	`value: ${{ jobs.x.outputs.y }}`
Visibility in UI	Expands steps inline in parent job	Separate job entries in the workflow run
Nesting	Can call other composite actions	Can call other reusable workflows (max 4 deep)
File location	Any repo, any path	`.github/workflows/` only
Cross-platform	Yes (shares caller's runner OS)	Per runner in reusable workflow
Parallel jobs within	❌ No	✅ Yes

Dimension	Composite Action	Reusable Workflow
Best for	Step sequence reuse	Job/pipeline template reuse
Jenkins equivalent	Shared Library function	Shared Library pipeline template
Startup overhead	None (inline)	Runner provisioning time per job

Visual Decision Tree

```
Do you need to reuse something?
│
├── Is it a sequence of STEPS that should run within an existing job?
│   └── YES → Composite Action
│       Examples:
│         • "Set up Go + cache + download modules"
│         • "Build Docker image + push to registry + output digest"
│         • "Run Trivy scan + upload SARIF + write summary"
│
└── Is it an entire JOB or PIPELINE that should run independently?
    └── YES → Reusable Workflow
        Examples:
          • "Full CI pipeline: lint → test → coverage → sonar"
          • "Deploy to an environment with approval gate"
          • "Build → test → publish across a matrix of OS/versions"
```

If you're still unsure, ask these questions:

```
Does it need its own runner?        YES → Reusable Workflow
Does it need sidecar services?      YES → Reusable Workflow
Does it need environment approvals? YES → Reusable Workflow
Does it need parallel internal jobs?YES → Reusable Workflow
Is it just steps I repeat often?    YES → Composite Action
Does it need to run on the caller's filesystem without re-checkout? YES →
Composite Action
```

Concrete Examples That Clarify the Distinction

Example A — Setup step: Composite Action is right

```yaml
# ✅ Composite Action — this is a STEP sequence, not a job
# .github/actions/setup-python-env/action.yml
runs:
  using: composite
  steps:
    - uses: actions/setup-python@v5
```

```
      with:
        python-version: ${{ inputs.python-version }}
    - uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
    - shell: bash
      run: pip install -r requirements.txt

# Usage in a workflow:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-python-env   # ← step-level call
        with:
          python-version: '3.12'
      - run: pytest                                  # ← continues on same
runner
```

Example B — Full CI pipeline: Reusable Workflow is right

```
# ✅ Reusable Workflow — this is a full JOB pipeline
# platform-workflows/.github/workflows/python-ci.yml
on:
  workflow_call:
    inputs:
      python-version:
        type: string
        default: '3.12'
      run-integration-tests:
        type: boolean
        default: false

jobs:
  lint:                           # ← own runner
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-python-env
      - run: flake8 . && mypy .

  test:                           # ← own runner, runs parallel to lint
    runs-on: ubuntu-latest
    services:
      postgres:                   # ← service container — only possible in
reusable workflow
        image: postgres:15
        env:
          POSTGRES_PASSWORD: test
```

```
        ports:
          - 5432:5432
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-python-env
      - run: pytest --cov=src

  integration:                    # ← own runner, conditional
    runs-on: ubuntu-latest
    needs: test
    if: inputs.run-integration-tests == true
    steps:
      - uses: actions/checkout@v4
      - run: pytest tests/integration/
```

Example C — Where the wrong choice causes problems

```
# ❌ WRONG: Trying to use composite action for something needing services
# This WILL NOT WORK — composite actions cannot define services

# action.yml
runs:
  using: composite
  steps:
    # ERROR: You cannot spin up a postgres sidecar here
    # The caller's job would need to declare the service
    - shell: bash
      run: pytest tests/integration/

# ✅ CORRECT: Use a reusable workflow, which CAN define services
# platform/.github/workflows/integration-tests.yml
on:
  workflow_call:
jobs:
  integration:
    runs-on: ubuntu-latest
    services:                     # ← only possible here
      postgres:
        image: postgres:15
    steps:
      - run: pytest tests/integration/
```

The Layered Architecture — Using Both Together

The most powerful pattern is using **both** in a layered way:

```
Layer 1: Reusable Workflows (job/pipeline level)
    ┌──────────────────────────────────────────────────────────┐
```

```
      │ reusable-deploy.yml                          │
      │   Job: validate                              │
      │   Job: deploy ──────────────────────────┐    │
      │     Step: uses: ./.github/actions/      │    │
      │           k8s-deploy ◄── Composite      │    │
      │     Step: uses: ./.github/actions/      │    │
      │           notify-slack ◄── Composite    │    │
      │   Job: verify                           │    │
      └─────────────────────────────────────────┘    │
                                                     │
  Layer 2: Composite Actions (step level)            │
      ┌──────────────────────────────────────────────▼──┐
      │ .github/actions/k8s-deploy/action.yml           │
      │   Step: configure-aws-oidc                      │
      │   Step: update-kubeconfig                       │
      │   Step: helm-upgrade                            │
      │   Step: verify-rollout                          │
      └─────────────────────────────────────────────────┘
```

```yaml
# The reusable workflow uses the composite action internally
# platform-workflows/.github/workflows/reusable-deploy.yml
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    steps:
      - uses: actions/checkout@v4

      # Using a composite action FROM WITHIN a reusable workflow
      - uses: my-org/platform-actions/.github/actions/k8s-deploy@v2
        with:
          cluster: ${{ inputs.cluster-name }}
          image-tag: ${{ inputs.image-tag }}
          aws-role-arn: ${{ secrets.aws-role-arn }}
```

Interview Answer (30–45 seconds)

> "The core distinction is the execution level. A composite action is called at the step level and runs inline on the caller's existing runner — it shares the runner's filesystem, environment, and lifecycle. A reusable workflow is called at the job level and spins up its own runners — each job in the reusable workflow gets a fresh, independent runner. Reusable workflows can define services, matrix strategies, environments with approval gates, and parallel internal jobs — composite actions cannot. Choose composite when you're reusing a step sequence within a job. Choose reusable workflow when you're reusing an entire job or pipeline. In production, the best architectures layer both: reusable workflows for pipeline structure, composite actions for step-level tasks within those workflows."

Gotchas ⚠

- **A job using `uses:` cannot also have `runs-on:` or `steps:`.** These are mutually exclusive. If you put `uses:` at the job level, that's the entire job definition.
- **Composite actions DO share the caller's `GITHUB_TOKEN`.** If the caller has `contents: write`, the composite action's steps have it too. If the caller has `contents: read`, the composite action is restricted to read. You cannot escalate permissions inside a composite action.
- **Reusable workflows can set their own `permissions:`.** This is a key security feature — the reusable workflow's jobs run with their declared permissions regardless of what the caller has. This is both a feature (scoped least-privilege) and a gotcha (caller can't grant more than GitHub allows for the event type).
- **Composite action outputs must map to step outputs.** They cannot return arbitrary computed values — only values that were previously written to `$GITHUB_OUTPUT` in a step with an `id:`.
- **Step names in composite actions appear in the UI under the parent step's name.** They're visible and individually statused, but grouped under the composite action's name — good for debugging.

3.4 Workflow Templates — Org-Level `.github` Repo

What It Is

A **workflow template** is a pre-written workflow YAML file stored in an org's special `.github` repository that appears as a template option in the Actions UI when setting up new workflows in any repo in that org.

Unlike reusable workflows (which are called programmatically), workflow templates are **starting point files** — they're copied into the target repo when a team selects them from the UI. They are not dynamically linked.

Jenkins equivalent: The Jenkins pipeline wizard templates, or Jenkinsfile templates you paste from a wiki. The difference: GitHub integrates these templates directly into the repo setup UI — no wiki-hunting needed.

Why Workflow Templates Exist

Problem: Even with reusable workflows available, new repos still need a starting workflow file. Without templates, every team:

1. Googles "GitHub Actions YAML"
2. Finds a random blog post example
3. Copies it and adapts it — inconsistently
4. Misses security best practices
5. Doesn't know about your org's platform workflows

Workflow templates solve the **onboarding problem** — they ensure new repos start with a best-practice workflow that already calls your reusable workflows and follows org standards.

How It Works Internally

```
Org has: my-org/.github  ← special org-level repo

my-org/.github/
  workflow-templates/
    node-service.yml           ← Template YAML
    node-service.properties.json ← Template metadata
    python-service.yml
    python-service.properties.json
    terraform.yml
    terraform.properties.json

When any repo in my-org creates a new workflow:
  Actions tab → New workflow → "By my-org" section shows these templates

User clicks template → YAML is COPIED into repo's .github/workflows/
(It's a copy — not a live link to the template)
```

Creating a Workflow Template — Complete Example

Step 1: The Template YAML

```yaml
# my-org/.github/workflow-templates/node-service.yml

name: Node.js Service CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # ── CI: runs our org standard CI pipeline ──────────────────────
  ci:
    uses: my-org/platform-workflows/.github/workflows/reusable-ci.yml@v3
    with:
      service-name: ${{ github.event.repository.name }}
      node-version: '20'
      # CUSTOMIZE: Set your coverage threshold
      coverage-threshold: 80
    secrets:
      sonar-token: ${{ secrets.SONAR_TOKEN }}

  # ── Build Docker image ─────────────────────────────────────────
  build:
    needs: ci
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
```

```yaml
    uses: my-org/platform-workflows/.github/workflows/reusable-docker-
build.yml@v3
    with:
      # CUSTOMIZE: Update image name to match your service
      image-name: ${{ github.event.repository.name }}
      registry: ghcr.io/my-org
      tag: ${{ github.sha }}
    secrets:
      registry-token: ${{ secrets.GITHUB_TOKEN }}

  # ── Deploy to staging ──────────────────────────────────────────
  deploy-staging:
    needs: build
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@v3
    with:
      environment: staging
      image-tag: ${{ github.sha }}
      # CUSTOMIZE: Update helm-release-name to match your service
      helm-release-name: ${{ github.event.repository.name }}
    secrets:
      kubeconfig-base64: ${{ secrets.STAGING_KUBECONFIG_B64 }}
      slack-webhook: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}

  # ── Deploy to production (requires approval) ──────────────────
  deploy-production:
    needs: deploy-staging
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    uses: my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@v3
    with:
      environment: production
      image-tag: ${{ github.sha }}
      helm-release-name: ${{ github.event.repository.name }}
    secrets:
      kubeconfig-base64: ${{ secrets.PROD_KUBECONFIG_B64 }}
      slack-webhook: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}
```

Step 2: The Properties File

```jsonc
// my-org/.github/workflow-templates/node-service.properties.json
{
  "name": "Node.js Service Pipeline",
  "description": "Standard CI/CD pipeline for Node.js microservices.
Includes lint, test, Docker build, staging deploy, and production deploy
with approval gate.",
  "iconName": "node",
  "categories": [
    "Continuous integration",
    "Deployment"
```

```
  ]
}
```

`iconName` **options:** Uses Octicons names. Common ones: `node`, `python`, `go`, `ruby`, `java`, `docker`, `tools`, `code`, `rocket`, `shield`.

Step 3: Multiple Templates for Different Service Types

```
my-org/.github/workflow-templates/
  ├── node-service.yml
  ├── node-service.properties.json
  ├── python-service.yml
  ├── python-service.properties.json
  ├── go-service.yml
  ├── go-service.properties.json
  ├── terraform-module.yml
  ├── terraform-module.properties.json
  ├── library-publish.yml            ← For npm/PyPI packages
  ├── library-publish.properties.json
  └── security-scan-only.yml         ← For repos that just need scanning
      security-scan-only.properties.json
```

What Appears in the UI

When any repo in `my-org` navigates to Actions → New workflow:

```
┌──────────────────────────────────────────────────────────────┐
│  Choose a workflow                                            │
│                                                               │
│  By my-org                                                    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 🟢 Node.js Service Pipeline                          │    │
│  │    Standard CI/CD pipeline for Node.js microservices│    │
│  │    [Configure]                                       │    │
│  ├──────────────────────────────────────────────────────┤    │
│  │ 🐍 Python Service Pipeline                           │    │
│  │    Standard CI/CD for Python microservices           │    │
│  │    [Configure]                                       │    │
│  ├──────────────────────────────────────────────────────┤    │
│  │ 🏗️  Terraform Module CI                              │    │
│  │    Validates, plans, and applies Terraform modules   │    │
│  │    [Configure]                                       │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                               │
│  Suggested for this repository                                │
│  [GitHub's marketplace suggestions...]                        │
└──────────────────────────────────────────────────────────────┘
```

Key Limitation: Templates Are Copies, Not Live Links

```
Template updated in my-org/.github:
  v1: uses: platform-workflows/.github/workflows/deploy.yml@v2
  v2: uses: platform-workflows/.github/workflows/deploy.yml@v3  ← Security
fix

Impact: ZERO on existing repos that copied the template at v1.
        They still reference @v2 until a human updates them.

This is why templates alone are not enough for org-wide governance.
Reusable workflows (called programmatically) are the real mechanism.
Templates are for onboarding only.
```

Interview Answer (30–45 seconds)

> "Workflow templates live in an org's special `.github` repository under `workflow-templates/`. Each template is a YAML file paired with a JSON properties file defining its name and description. When any repo in the org creates a new workflow, GitHub shows these templates in the Actions UI. The important caveat is that templates are copied into the target repo — they're not dynamically linked. So a template update doesn't affect repos that already copied it. Templates solve the onboarding problem — new repos start with a best-practice workflow that already references your reusable workflows — but they're not a governance mechanism. For ongoing governance, you need reusable workflows or GitHub Enterprise's required workflows feature."

Gotchas ⚠

- **The `.github` repo must be public** for templates to be visible to all org members. If it's private, only org members with read access to that specific repo see the templates.
- **Template YAML is not validated before display** in the UI — a broken template YAML will be shown to users and only fail when they try to use it.
- **The `iconName` must exactly match an Octicons icon name** — if it doesn't, the template shows with no icon (not an error, just ugly).
- **Templates don't automatically use any org-level secrets.** The template references `${{ secrets.SONAR_TOKEN }}` — the team using the template must separately configure that secret in their repo or ensure it's an org-level secret.
- **Categories in the properties file are limited to GitHub's predefined list.** Custom categories are silently ignored.

3.5 Centralized Pipeline Management at Org Scale

What It Is

A systematic approach to owning, governing, and evolving CI/CD pipelines across an entire GitHub organization — typically owned by a Platform Engineering or Developer Experience team.

Jenkins equivalent: Running a central Jenkins instance with organization-level Shared Libraries, global tool configurations, and RBAC policies. The difference: GitHub Actions distributes the execution (each repo runs its own workflows) while centralizing the *definitions* and *standards* in dedicated repos.

Why This Matters

Without centralized management at scale:

```
PROBLEM SPACE (50+ repos without governance):
  ✗ 50 different versions of the deploy workflow
  ✗ Some repos still use aws-actions/configure-aws-credentials@v1 (has
security issues)
  ✗ 12 different caching strategies — most are wrong
  ✗ No enforcement of security scanning
  ✗ No consistent environment protection rules
  ✗ When you change the Kubernetes deploy process, 50 PRs required
  ✗ New repos take 2 days to set up a working pipeline
  ✗ Impossible to audit what's running where

WITH CENTRALIZED MANAGEMENT:
  ✓ One reusable deploy workflow — 50 callers
  ✓ Action version updates happen in one PR
  ✓ Security scanner is mandatory — enforced via required workflows
  ✓ New repos have a working pipeline in 10 minutes via templates
  ✓ Audit: query which repos call which version of which workflow
```

The Full Architecture

```
my-org/
│
├── .github/                          ← Org-level special repo
│   ├── workflow-templates/           ← Onboarding templates
│   │   ├── node-service.yml
│   │   ├── python-service.yml
│   │   └── terraform-module.yml
│   └── SECURITY.md
│
├── platform-workflows/               ← Platform team owns
│   ├── .github/
│   │   └── workflows/
│   │       ├── reusable-ci.yml
│   │       ├── reusable-docker-build.yml
```

```
│   │       ├── reusable-deploy.yml
│   │       ├── reusable-security-scan.yml
│   │       ├── reusable-terraform.yml
│   │       └── reusable-release.yml
│   └── README.md
│
├── platform-actions/                ← Composite actions repo
│   └── .github/
│       └── actions/
│           ├── k8s-deploy/
│           ├── setup-go/
│           ├── notify-slack/
│           └── oidc-aws-auth/
│
├── payment-api/                     ← Product teams own
│   └── .github/workflows/
│       └── main.yml  ← calls platform-workflows
│
├── user-service/
│   └── .github/workflows/
│       └── main.yml
│
└── 47 more service repos...
```

Layer 1: The Platform Workflows Repository

```yaml
# platform-workflows/.github/workflows/reusable-security-scan.yml
# Mandatory security scanning workflow — called by ALL service pipelines
name: Reusable — Security Scan

on:
  workflow_call:
    inputs:
      image-to-scan:
        description: 'Docker image reference to scan (optional)'
        required: false
        type: string
        default: ''
      scan-filesystem:
        description: 'Scan the repository filesystem'
        required: false
        type: boolean
        default: true
      fail-on-severity:
        description: 'Fail if vulnerabilities of this severity or higher
are found'
        required: false
        type: string
        default: 'CRITICAL'
    secrets:
```

```yaml
      github-token:
        required: true
    outputs:
      vulnerability-count:
        value: ${{ jobs.scan.outputs.count }}
      gate-passed:
        value: ${{ jobs.scan.outputs.passed }}

permissions:
  contents: read
  security-events: write    # For SARIF upload to GitHub Security tab

jobs:
  scan:
    name: Trivy Security Scan
    runs-on: ubuntu-latest
    outputs:
      count: ${{ steps.count.outputs.total }}
      passed: ${{ steps.gate.outputs.passed }}

    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2

      - name: Scan filesystem
        if: inputs.scan-filesystem == true
        uses: aquasecurity/trivy-
action@915b19bbe73b92a6cf82a1bc12b087c9a19a5fe  # v0.28.0
        with:
          scan-type: fs
          scan-ref: .
          format: sarif
          output: trivy-fs-results.sarif
          severity: ${{ inputs.fail-on-severity }},HIGH,MEDIUM
          ignore-unfixed: true

      - name: Scan Docker image
        if: inputs.image-to-scan != ''
        uses: aquasecurity/trivy-
action@915b19bbe73b92a6cf82a1bc12b087c9a19a5fe
        with:
          scan-type: image
          image-ref: ${{ inputs.image-to-scan }}
          format: sarif
          output: trivy-image-results.sarif
          severity: ${{ inputs.fail-on-severity }}
          exit-code: 1

      - name: Upload SARIF to GitHub Security
        if: always()
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: .
        env:
```

```
          GITHUB_TOKEN: ${{ secrets.github-token }}

      - name: Count vulnerabilities
        id: count
        if: always()
        run: |
          TOTAL=0
          for f in trivy-*-results.sarif; do
            if [ -f "$f" ]; then
              COUNT=$(jq '[.runs[].results | length] | add // 0' "$f")
              TOTAL=$((TOTAL + COUNT))
            fi
          done
          echo "total=${TOTAL}" >> $GITHUB_OUTPUT

      - name: Evaluate security gate
        id: gate
        if: always()
        run: |
          COUNT=${{ steps.count.outputs.total }}
          if [ "$COUNT" -gt 0 ] && [ "${{ inputs.fail-on-severity }}" !=
"" ]; then
            echo "passed=false" >> $GITHUB_OUTPUT
          else
            echo "passed=true" >> $GITHUB_OUTPUT
          fi
```

Layer 2: Required Workflows (GitHub Enterprise) ⚠

Required workflows are the **enforcement mechanism** — they run automatically on all repos in an org, regardless of what those repos declare in their own workflow files.

```
# Org Settings → Actions → Required Workflows → Add workflow

# Configuration (in GitHub UI or via API):
# Repository: my-org/platform-workflows
# Workflow path: .github/workflows/required-security-scan.yml
# Target: All repositories (or selected repos/topics)
# Ref: @main (or a specific tag)

# required-security-scan.yml — this runs on EVERY repo's push/PR
name: Required — Security Scan
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  security:
```

```
    uses: my-org/platform-workflows/.github/workflows/reusable-security-
scan.yml@v3
    with:
      scan-filesystem: true
      fail-on-severity: CRITICAL
    secrets:
      github-token: ${{ secrets.GITHUB_TOKEN }}
```

What required workflows mean in practice:

- They appear as additional status checks on every PR in every targeted repo
- Branch protection rules can require them to pass before merge
- Teams **cannot** disable or skip them — they're org-enforced
- If the required workflow fails, the PR cannot merge (if branch protection is configured)

Layer 3: Governance and Observability

Audit Which Repos Call Which Workflow Version

```
# Using GitHub CLI to audit workflow usage across the org
gh api graphql -f query='
{
  organization(login: "my-org") {
    repositories(first: 100) {
      nodes {
        name
        object(expression: "HEAD:.github/workflows") {
          ... on Tree {
            entries {
              name
              object {
                ... on Blob {
                  text
                }
              }
            }
          }
        }
      }
    }
  }
}'

# Or using the REST API to find all workflow files referencing a specific
workflow
gh api /orgs/my-org/repos --paginate --jq '.[].name' | while read repo; do
  gh api /repos/my-org/$repo/contents/.github/workflows --jq '.[].name'
2>/dev/null | \
    while read wf; do
      CONTENT=$(gh api /repos/my-org/$repo/contents/.github/workflows/$wf
```

```
--jq '.content' | base64 -d)
      if echo "$CONTENT" | grep -q "platform-workflows"; then
        echo "$repo/$wf: $(echo "$CONTENT" | grep 'platform-workflows' |
head -1)"
      fi
    done
done
```

Automated Version Drift Detection

```
# platform-workflows/.github/workflows/audit-caller-versions.yml
# Runs weekly to report which repos are using outdated workflow versions
name: Audit Caller Workflow Versions

on:
  schedule:
    - cron: '0 9 * * 1'  # Monday 9 AM UTC
  workflow_dispatch:

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - name: Find repos using old workflow versions
        env:
          GH_TOKEN: ${{ secrets.ORG_READ_TOKEN }}
        run: |
          CURRENT_VERSION="v3"
          OUTDATED_REPOS=()

          while IFS= read -r repo; do
            # Check if this repo calls our platform workflows
            WORKFLOWS=$(gh api /repos/my-
org/$repo/contents/.github/workflows \
              --jq '.[].name' 2>/dev/null || continue)

            for wf in $WORKFLOWS; do
              CONTENT=$(gh api /repos/my-
org/$repo/contents/.github/workflows/$wf \
                --jq '.content' 2>/dev/null | base64 -d 2>/dev/null)

              # Check for old version references
              if echo "$CONTENT" | grep -q "platform-workflows" && \
                 ! echo "$CONTENT" | grep -q "@${CURRENT_VERSION}"; then
                OUTDATED_REPOS+=("$repo/$wf")
              fi
            done
          done < <(gh api /orgs/my-org/repos --paginate --jq '.[].name')

          if [ ${#OUTDATED_REPOS[@]} -gt 0 ]; then
            echo "## ⚠ Repos using outdated platform workflow versions" >>
```

```
$GITHUB_STEP_SUMMARY
            printf '%s\n' "${OUTDATED_REPOS[@]}" | \
              awk '{print "- "$0}' >> $GITHUB_STEP_SUMMARY
          else
            echo "## ✅ All repos using current platform workflow version"
>> $GITHUB_STEP_SUMMARY
          fi
```

Layer 4: Self-Service Onboarding

```
# platform-workflows/.github/workflows/onboard-new-service.yml
# Triggered via repository_dispatch when a new service repo is created
name: Onboard New Service

on:
  repository_dispatch:
    types: [service-repo-created]
  workflow_dispatch:
    inputs:
      repo-name:
        required: true
        type: string
      service-type:
        required: true
        type: choice
        options: [node, python, go, terraform]
      team-slug:
        required: true
        type: string

jobs:
  onboard:
    runs-on: ubuntu-latest
    steps:
      - name: Get service config
        id: config
        run: |
          REPO="${{ github.event.inputs.repo-name ||
github.event.client_payload.repo }}"
          TYPE="${{ github.event.inputs.service-type ||
github.event.client_payload.type }}"
          echo "repo=${REPO}" >> $GITHUB_OUTPUT
          echo "type=${TYPE}" >> $GITHUB_OUTPUT

      - name: Create workflow from template
        env:
          GH_TOKEN: ${{ secrets.PLATFORM_BOT_TOKEN }}
        run: |
          REPO="${{ steps.config.outputs.repo }}"
          TYPE="${{ steps.config.outputs.type }}"
```

```
          # Get template content
          TEMPLATE=$(gh api \
            /repos/my-org/.github/contents/workflow-templates/${TYPE}-
service.yml \
            --jq '.content' | base64 -d)

          # Create workflow file in new repo
          gh api \
            --method PUT \
            /repos/my-org/$REPO/contents/.github/workflows/main.yml \
            -f message="chore: add platform CI/CD workflow" \
            -f content="$(echo "$TEMPLATE" | base64 -w 0)"

      - name: Configure branch protection
        env:
          GH_TOKEN: ${{ secrets.PLATFORM_BOT_TOKEN }}
        run: |
          REPO="${{ steps.config.outputs.repo }}"
          gh api \
            --method PUT \
            /repos/my-org/$REPO/branches/main/protection \
            -f required_status_checks='{"strict":true,"contexts":
["ci","security-scan"]}' \
            -f enforce_admins=false \
            -f
required_pull_request_reviews='{"required_approving_review_count":1}' \
            -f restrictions=null

      - name: Configure org-level secrets access
        env:
          GH_TOKEN: ${{ secrets.PLATFORM_ADMIN_TOKEN }}
        run: |
          REPO="${{ steps.config.outputs.repo }}"
          # Grant access to org secrets needed by the pipeline
          for secret in SONAR_TOKEN SLACK_DEPLOY_WEBHOOK
STAGING_KUBECONFIG_B64; do
            gh secret share $secret --repo my-org/$REPO
          done

      - name: Notify team
        run: |
          echo "New service ${{ steps.config.outputs.repo }} onboarded
successfully" \
            | gh workflow run notify-team.yml \
              --field message="$(cat -)"
```

Layer 5: Cost and Usage Visibility

```
# GitHub Actions usage by repo (via API)
gh api /orgs/my-org/settings/billing/actions --jq '{
  total_minutes_used: .total_minutes_used,
  paid_minutes: .total_paid_minutes_used,
  breakdown: .minutes_used_breakdown
}'

# Per-repo usage (requires org owner PAT)
gh api /repos/my-org/payment-api/actions/runs \
  --paginate \
  --jq '[.workflow_runs[] | {
    workflow: .name,
    status: .conclusion,
    duration_ms: (.updated_at | fromdate) - (.created_at | fromdate),
    runner_type: .runner_name
  }]' | jq 'group_by(.workflow) | map({
    workflow: .[0].workflow,
    runs: length,
    avg_duration_s: (map(.duration_ms) | add / length)
  })'
```

The Org-Scale Governance Matrix

Mechanism	What it Enforces	Owner	Effort
Workflow Templates	Starting point for new repos	Platform	Low
Reusable Workflows (versioned)	Shared pipeline logic	Platform	Medium
Required Workflows (GHES)	Mandatory scans/checks on all PRs	Platform	Medium
Branch Protection Rules	Required status checks, review counts	Platform/Repo	Low
Org-Level Secrets	Shared credentials (no per-repo setup)	Platform	Low
Org Action Policies	Allowlist approved actions	Security	Medium
Dependabot on Platform Repo	Keep action SHAs current	Platform	Low
Weekly Audit Workflow	Detect version drift	Platform	Medium
OIDC + IAM Conditions	Zero long-lived credentials	Security	High
CODEOWNERS on Platform Repo	Require platform review on changes	Platform	Low

---

Interview Answer (30–45 seconds)

> "Org-scale pipeline management is a system built on layered GitHub Actions features. The foundation is a platform-workflows repo owned by the platform team, containing versioned reusable workflows that all service repos call. On top of that, workflow templates in the org's `.github` repo give new services a working pipeline in minutes. For mandatory compliance — security scanning, required checks — GitHub Enterprise's required workflows enforce specific workflows on all PRs across all repos, regardless of what individual repos declare. Org-level secrets and OIDC roles eliminate per-repo credential management. The platform team owns one repo, and governance changes propagate to all callers through versioned updates and — for required workflows — automatically."

Deep Dive: Migration Strategy from Jenkins at Org Scale

```
Phase 1: Inventory (Week 1-2)
  - Catalog all Jenkins pipelines
  - Identify common patterns (build, test, deploy, scan)
  - Map Jenkins Shared Library functions → composite actions
  - Map Jenkins pipeline templates → reusable workflows

Phase 2: Platform Infrastructure (Week 3-4)
  - Create platform-workflows repo with initial reusable workflows
  - Create platform-actions repo with composite actions
  - Set up OIDC for all cloud providers (no more Jenkins credentials)
  - Create workflow templates in org .github repo

Phase 3: Pilot Migration (Week 5-8)
  - Migrate 3-5 low-risk services to GitHub Actions
  - Refine reusable workflows based on real usage
  - Document gotchas, onboarding guide

Phase 4: Scaled Migration (Week 9-20)
  - Team-by-team migration using onboarding automation
  - Run Jenkins and GitHub Actions in parallel during transition
  - Use branch protection to enforce GitHub Actions checks

Phase 5: Decommission (Week 21+)
  - Disable Jenkins pipelines for migrated repos
  - Keep Jenkins for any repos with blockers
  - Set up required workflows for remaining compliance checks
```

Common Interview Questions

Q: How do you prevent teams from bypassing your platform workflows and writing their own?

> "Multiple layers. First, GitHub Enterprise's required workflows enforce specific workflows on every PR in every repo — they can't be opted out of. Second, org-level action policies allowlist only approved action sources so teams can't pull in arbitrary actions. Third, OIDC credentials are only configured for specific IAM roles that match the reusable workflow's `job_workflow_ref` claim —

> so a custom workflow can't assume the deployment role. Fourth, branch protection rules require the required workflow checks to pass. Finally, CODEOWNERS on the platform-workflows repo means any changes to shared pipelines require platform team review. No single mechanism is sufficient — defense in depth."

Q: How do you update your reusable workflow across 50 calling repos when you have a breaking change?

> "For non-breaking changes, teams on the floating `@v3` tag automatically get them. For breaking changes, I version the major: release `@v4` with the breaking change, keep `@v3` alive in maintenance mode, communicate the migration with a changelog and migration guide, then use our audit workflow to track adoption. I'd also write a codemod or script to automate the `sed` changes in calling workflows if the migration is mechanical. GitHub Enterprise's required workflows give me an escape hatch — I can force the new version via required workflow while giving teams time to update their calling code."

Gotchas ⚠

- **`secrets: inherit` at org scale is a security risk.** If you use `secrets: inherit` in your reusable workflow callers, any secret in any calling repo is accessible to the reusable workflow. Prefer explicit secret declarations.
- **Required workflows run against the BASE branch's workflow definition.** If you push a required workflow change to `main`, it takes effect immediately for all future PRs — even open PRs suddenly have a new required check. Plan required workflow releases carefully.
- **Reusable workflow `@main` in a required workflow means every push to `main` of the platform repo changes required checks org-wide in real time.** Always pin required workflows to a stable tag, not `@main`.
- **CODEOWNERS on the platform repo is non-negotiable.** Without it, any org member who can write to the platform repo can change shared pipelines for everyone.
- **Org-level secrets have a 100-secret limit.** Plan secret naming and scope carefully. Group secrets by environment (`STAGING_*`, `PROD_*`) and by purpose.
- **The GitHub Actions API for audit has rate limits.** The org-wide audit script above will hit secondary rate limits for large orgs (500+ repos). Add `gh api --paginate` and `sleep` between requests, or use GraphQL with pagination for efficiency.

Cross-Topic Interview Questions

These span the full Category 3 and are realistic senior/staff interview questions.

Q: You're leading a platform team at a company with 80 repos. Design your CI/CD platform architecture using GitHub Actions. Walk me through every layer.

> "Layer one: a `platform-workflows` repo containing reusable workflows versioned with semantic tags — reusable-ci, reusable-docker-build, reusable-deploy, reusable-security-scan, reusable-terraform. These are the core pipeline primitives. Layer two: a `platform-actions` repo with

> composite actions for common step sequences — OIDC auth setup, Helm deploy, Slack notifications, Docker build+push. Reusable workflows call these composite actions internally. Layer three: an org `.github` repo with workflow templates — one per service type: Node, Python, Go, Terraform. Templates pre-wire calls to the reusable workflows. Layer four: for compliance, GitHub Enterprise required workflows — mandatory security scanning on every PR, can't be opted out. Layer five: org-level secrets and OIDC with IAM role conditions scoped to specific reusable workflow refs. Layer six: a weekly audit workflow that reports which repos are on outdated platform workflow versions and auto-creates upgrade PRs."

Q: A developer asks why their workflow can't access secrets they see declared in the reusable workflow's `workflow_call.secrets` block. What's wrong?

> "The `workflow_call.secrets` declaration says what the reusable workflow *expects to receive* — it's an interface definition, not a secret store. The actual secret value must be passed by the caller via the `secrets:` key in its `uses:` job. If the caller doesn't pass it, the secret is empty in the reusable workflow regardless of the declaration. Check the calling workflow: either add `secrets: my-secret: ${{ secrets.MY_SECRET }}` explicitly, or use `secrets: inherit` to pass all caller secrets. Also verify the secret is actually configured in the caller's repo or as an org secret the repo can access."

Q: Composite action vs reusable workflow — give me an example where using the wrong one causes a real production problem.

> "Classic mistake: team builds a composite action for their integration test suite that needs a Postgres sidecar. Composite actions cannot declare services — they inherit the caller's job. So the composite action's test steps try to connect to `localhost:5432`, but there's no Postgres running because the caller's job didn't declare it. The fix requires either: moving to a reusable workflow that can declare services, OR having the caller job declare the service and the composite action just runs the tests. Another gotcha in reverse: a team uses a reusable workflow when they just need a composite action — now every CI run provisions two extra runners (one for the reusable workflow's jobs) costing real money and adding 45 seconds of runner startup per run."

Q: How do you handle a situation where different teams need slightly different behavior in a shared deploy workflow?

> "Several strategies depending on how different. If it's parameter variation — replica counts, timeout values, image registries — typed inputs handle it. If it's structural variation — some teams need a canary step, some don't — use boolean inputs that gate entire job sections with `if: inputs.enable-canary == true`. If teams need fundamentally different deploy strategies, consider whether you have two distinct deploy patterns that each deserve their own reusable workflow rather than one workflow trying to cover all cases with a boolean explosion. The signal that you've over-parameterized is when the reusable workflow has 20+ inputs and becomes impossible to reason about — at that point, split into specialized workflows and use workflow templates to point teams to the right one for their service type."

Quick Reference Cheat Sheet

Reusable Workflow Skeleton

```yaml
# THE DEFINITION (in platform repo)
on:
  workflow_call:
    inputs:
      my-string:    { type: string,  required: true }
      my-bool:      { type: boolean, required: false, default: false }
      my-number:    { type: number,  required: false, default: 3 }
    secrets:
      my-secret:    { required: true }
      optional-sec: { required: false }
    outputs:
      result:
        value: ${{ jobs.my-job.outputs.result }}
        description: 'The result'

jobs:
  my-job:
    runs-on: ubuntu-latest
    environment: ${{ inputs.my-string }}   # Can target environments!
    outputs:
      result: ${{ steps.work.outputs.result }}
    steps:
      - id: work
        run: |
          echo "Input: ${{ inputs.my-string }}"
          echo "Secret present: ${{ secrets.my-secret != '' }}"
          echo "result=done" >> $GITHUB_OUTPUT
```

```yaml
# THE CALL (in app repo)
jobs:
  call-it:
    uses: org/platform/.github/workflows/my-workflow.yml@v3
    with:
      my-string: production
      my-bool: true
    secrets:
      my-secret: ${{ secrets.MY_SECRET }}
      # OR: secrets: inherit

  use-outputs:
    needs: call-it
    runs-on: ubuntu-latest
    steps:
      - run: echo "${{ needs.call-it.outputs.result }}"
```

Decision Matrix

```
Need to reuse...         →  Use...
──────────────────────────────────────────────────────
Steps within a job       →  Composite Action
A full job/pipeline      →  Reusable Workflow
Services (DB, Redis)     →  Reusable Workflow (only option)
Environment approvals    →  Reusable Workflow (only option)
Matrix strategy          →  Reusable Workflow (only option)
Step sequence, no runner →  Composite Action (faster)
Org-wide starting point  →  Workflow Template (copy, not linked)
Mandatory org-wide check →  Required Workflow (GHES only)
```

Workflow Template File Pair

```
my-org/.github/workflow-templates/
  service-type.yml                  ← The YAML template
  service-type.properties.json      ←
{"name":"...","description":"...","iconName":"...","categories":[...]}
```

Secret Passing Summary

```yaml
# Explicit (recommended for production — auditable)
secrets:
  deploy-key: ${{ secrets.DEPLOY_KEY }}
  api-token: ${{ secrets.API_TOKEN }}

# Inherit all (convenient, less auditable)
secrets: inherit

# Checking optional secret in reusable workflow
- if: secrets.optional-token != ''
  run: ./use-token.sh
  env:
    TOKEN: ${{ secrets.optional-token }}
```

Cross-Repo `uses:` Formats

```yaml
# Same repo
uses: ./.github/workflows/reusable.yml

# Cross-repo (different repo, same or different org)
uses: org/repo/.github/workflows/workflow.yml@ref

# Ref options:
```

```
@main            # branch (floating — dev/test only)
@v3              # floating major tag
@v3.1.0          # specific version
@a3f8c2d1e4b5    # immutable SHA (recommended for production)
```

Output Chain (Step → Job → workflow_call)

```
Step:  echo "key=val" >> $GITHUB_OUTPUT   (step id: my-step)
Job:   outputs:
         key: ${{ steps.my-step.outputs.key }}   (job id: my-job)
WFC:   outputs:
         key:
           value: ${{ jobs.my-job.outputs.key }}
Caller: ${{ needs.calling-job-id.outputs.key }}
```

End of Category 3: Reusability & Modularity

> **Next:** Category 4 (Secrets, Environments & Security) builds directly on the patterns here — especially OIDC scoped to specific `job_workflow_ref` claims, which is the enforcement mechanism that makes org-scale pipeline governance work from a credentials perspective.
>
> **Practice:** Model your current org's CI/CD in terms of these layers. Which parts would be reusable workflows? Which composite actions? Where would templates help? That mental mapping is what makes these answers concrete in interviews.